

Министерство образования и науки РФ
Санкт-Петербургский Политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

Отчёт по Курсовой работе №2

по дисциплине «Технологии разработки качественного программного обеспечения»

Выполнил:

студент гр. 5130904/10102

Богданов Н.Р.

Преподаватель:

Маслаков А. П.

Содержание

1	Введение	3
2	Используемые инструменты и методики	3
3	План выполнения интеграционных тестов	3
4	Интеграционные тесты	4
4.1	Авторизация пользователя	4
4.2	Регистрация пользователя	4
4.3	Загрузка аудио для обработки	5
4.4	Взятие списка транскрипций пользователя	7
4.5	Взятие статуса транскрипции	8
5	Отчёт о прохождении интеграционных тестов	9
6	Процедура расширения тестового набора	10

1. Введение

Цель документа:

Предоставить предварительное описание интеграционных тестов для проекта «Lazy-Lecture», включающее перечень тестовых сценариев, используемые инструменты и техники, а также план выполнения и дальнейшего расширения набора тестов. Документ предназначен для предварительного согласования у преподавателя.

Область применения:

Интеграционное тестирование охватывает проверку взаимодействия между ключевыми модулями и микросервисами проекта: *api*, *web-интерфейс*, а также взаимодействие с внешними системами (Postgres).

2. Используемые инструменты и методики

CI/CD: GitHub Actions — для автоматизации запуска интеграционных тестов по со-бытию изменения кода или по временным триггерам.

Mock-сервисы / Mock-объекты: Использование заглушек для имитации внешних сер-висов (например, БД Postgres, файловой системы, очередей) с целью изоляции тестовой среды.

3. План выполнения интеграционных тестов

1. Подготовка тестовой среды:

Формирование окружения с использованием mock-сервисов. Настройка CI-сервера (GitHub Actions).

2. Разработка тестовых сценариев:

Создание предварительного описания тестовых кейсов (как описано в разделе 2).
Утверждение сценариев у преподавателя.

3. Запуск и мониторинг тестов:

Автоматический запуск тестов по событию изменения кода или по расписанию. Сбор логов и первичный анализ результатов.

4. Отчетность:

Формирование отчёта о прохождении тестов (раздел 5 ниже) — заполняется после проведения тестирования.

4. Интеграционные тесты

4.1. Авторизация пользователя

SUT: Auth Service (`source.app.auth.services.authenticate_user`)

Downstream: База данных пользователей, PasswordHasher

Детальный сценарий: Функция получает имя пользователя и пароль, обращается к БД за записанным хэшем и вызывает проверку пароля. Если пользователь найден, активен и хэш совпадает, возвращается объект `User`; иначе — `None` или бросается исключение.

Сценарий 1 (успешный):

1. **Состояние до:** запись пользователя существует в БД, пользователь активен, пароль корректен.
2. **Состояние после:** возвращён объект `User` с тем же `id` и полями из БД.

Сценарий 2 (неуспешный — неверный пароль):

1. **Состояние до:** пользователь найден и активен, но передан неверный пароль.
2. **Состояние после:** возвращается `None`.

Сценарий 3 (неуспешный — неактивный пользователь):

1. **Состояние до:** пользователь найден, но поле `active=False`.
2. **Состояние после:** выбрасывается `HTTPException("Your account is blocked")`.

```
1 async def test_authorization_user(monkeypatch):
2     # Создаем фейкового пользователя с необходимыми
3     полями fake_user = User()
4     fake_user.id = 1
5     fake_user.username = "testuser"
6     fake_user.password = "hashedpassword"
7     fake_user.active = True fake_user.password_timestamp
8     = 1234567890.0
9
10    # Мокаем сессию БД: метод scalar возвращает fake_user
11    fake_db = AsyncMock()
12    fake_db.scalar = AsyncMock(return_value=fake_user)
13
14    # Патчим verify_password, чтобы он всегда возвращал True
15    monkeypatch.setattr("source.app.auth.services.verify_password", lambda
16    plain_password, hashed_password: True)
17
18    # Вызываем функцию аутентификации
19    result = await authenticate_user("testuser", "TestP@ss123", fake_db)
20    assert result is fake_user
```

4.2. Регистрация пользователя

SUT: User Service (`source.app.users.services.create_user`)

Downstream: База данных пользователей, PasswordHasher

Детальный сценарий: Функция получает объект `UserCreate`, хэширует пароль, со-здаёт запись пользователя в БД, коммитит и обновляет объект (присваивает `id`).

Если при

коммите возникает `IntegrityError` (дубликат), возвращается `None`.

Сценарий 1 (успешный):

1. **Состояние до:** в БД нет пользователя с таким `username`.
2. **Состояние после:** создан новый пользователь с `id=1`, пароль сохранён в хэше.

Сценарий 2 (неуспешный — дубликат):

1. **Состояние до:** в БД уже есть пользователь с тем же `username`.
2. **Состояние после:** в процессе `commit()` бросается `IntegrityError`, функция возвращает `None`.

```
1 async def test_registration_user(monkeypatch):
2     user_data = UserCreate(username="newuser", password="ValidP@ss123")
3     fake_db = AsyncMock()
4     # Мокаем методы работы с сессией: add, commit, refresh (refresh
    присваивает id)
5     fake_db.add = MagicMock()
6     fake_db.commit = AsyncMock()
7     fake_db.refresh = AsyncMock(side_effect=lambda user: setattr(user,
    "id"
8     , 1) or None)
9
10    result = await create_user(user_data, fake_db)
11    assert result is not None
12    assert result.username == "newuser"
13    # Пароль не должен совпадать с исходным он (
14    хешируется) assert result.password != "ValidP@ss123"
15    assert hasattr(result, "id") and result.id == 1
```

4.3. Загрузка аудио для обработки

SUT: Transcriptions View

(`source.app.transcriptions.views.create_upload_file`)

Downstream: Файловое хранилище (`aiofiles`), Queue Service, Transcriptions Service

Детальный сценарий: Принимается объект пользователя и файл, проверяется `can_interact`, читаются чанки через `aiofiles`, сохраняются на диск, вычисляется длительность аудио, создаётся запись транскрипции, отправляется задача в очередь.

Сценарий 1 (успешный):

1. **Состояние до:** в хранилище нет конфликтующих файлов, у пользователя `can_interact=True` очередь доступна.
2. **Состояние после:** файл сохранён по пути `/dummy_path/...`, создана транскрипция `id=42`, в очередь отправлено сообщение, возвращён JSON `{"message": "File uploaded successfully", "task_id": 42, ...}`.

Сценарий 2 (неуспешный — пользователь не может взаимодействовать):

1. **Состояние до:** у пользователя `can_interact=False`.
2. **Состояние после:** выбрасывается `HTTPException("Access restricted")`.

```

1 async def test_upload_audio(monkeypatch):
2     from source.app.transcriptions.views import create_upload_file
3
4     # Фейковый пользователь с правом can_interact
5     dummy_user = User()
6     dummy_user.id = 1
7     dummy_user.can_interact = True
8
9     # Фейковый объект аудиофайла с content_type "audio/mpeg"
10    class FakeUploadFile:
11        content_type = "audio/mpeg"
12
13        def _init_(self):
14            self.file = None # не используется в тесте
15            self._chunks = [b"audio_chunk", b""]
16            self._index = 0
17
18    async def read(self, size):
19        if self._index < len(self._chunks):
20            chunk = self._chunks[self._index]
21            self._index += 1
22            return chunk
23        return b""
24
25    fake_audio = FakeUploadFile()
26
27    # Фейковая очередь: кортеж канал(, имя очереди)
28    fake_channel = MagicMock()
29    fake_channel.basic_publish = MagicMock()
30    fake_task_q = (fake_channel, "dummy_queue")
31
32    # Мокаем работу с БД: get_current_transcriptions возвращает пустой
    список,
33    # а create_transcription возвращает фейковую транскрипцию с id=42.
34    fake_db = AsyncMock()
35    monkeypatch.setattr("source.app.transcriptions.views.
    get_current_transcriptions", AsyncMock(return_value=[]))
36    dummy_transcription = Transcription()
37    dummy_transcription.id = 42
38    dummy_transcription.creator_id = dummy_user.id
39    monkeypatch.setattr(
40        "source.app.transcriptions.views.create_transcription", AsyncMock(
    return_value=dummy_transcription)
41    )
42    # Патчим get_audio_duration, чтобы возвращалась фиксированная
    длительность например(, 120 секунд)
43    monkeypatch.setattr("source.app.transcriptions.views.get_audio_duration
    ", lambda filepath: 120.0)
44
45    # Патчим aiofiles.open, чтобы не производилось реальное обращение к
    файловой системе
46    class DummyAiofilesContextManager:
47        async def _aenter_(self):
48            return self
49
50        async def _aexit_(self, exc_type, exc, tb):
51            pass
52
53        async def write(self, data):
54            pass

```

```

55
56 monkeypatch.setattr(
57     "source.app.transcriptions.views.aiofiles.open", lambda filepath,
mode: DummyAiofilesContextManager()
58 )
59 # Патчим os.path.join для предсказуемого пути файла
60 monkeypatch.setattr(os.path, "join", lambda *parts: f"/dummy_path/{
parts[-1]}")
61
62 # Вызываем функцию загрузки аудио
63 result = await create_upload_file(dummy_user, fake_audio, fake_task_q,
fake_db)
64 assert result["message"] == "File uploaded successfully"
65 assert result["task_id"] == 42
66 assert "/dummy_path/" in result["file"]
67 # Проверяем, что функция отправки задания в очередь была вызвана
68 fake_channel.basic_publish.assert_called_once()
69

```

4.4. Взятие списка транскрипций пользователя

SUT: Transcriptions Service

(source.app.transcriptions.services.list_user_tran

Downstream: БД транскрипций, Description Provider

Детальный сценарий: Функция получает page, size, user_id, запрашивает общее число транскрипций (scalar) и страницу (scalars), для каждого результата вызывает полу-чение описания, и возвращает объект пагинации.

Сценарий 1 (есть транскрипции):

1. **Состояние до:** в БД есть 1 транскрипция для пользователя id=1.
2. **Состояние после:** возвращён объект с total=1, список из 1 элемента, у которого description="dummy description".

Сценарий 2 (нет транскрипций):

1. **Состояние до:** в БД нет транскрипций для user_id.
2. **Состояние после:** total=0, пустой список transcriptions.

```

1 async def test_list_user_transcriptions(monkeypatch):
2     from source.app.transcriptions.services import
3     list_user_transcriptions
4
5     # Создаем фейковый объект транскрипции
6     dummy_transcription = Transcription()
7     dummy_transcription.id = 101
8     dummy_transcription.creator_id = 1
9     dummy_transcription.audio_len_secs = 300.0
10    dummy_transcription.chunk_size_secs = 30.0
11    dummy_transcription.current_state = TranscriptionState.QUEUED
12    dummy_transcription.error_count = 0
13    dummy_transcription.create_date = datetime(2024, 1, 1)
14    dummy_transcription.update_date = datetime(2024, 1, 1)
15
16    # Мокаем сессию БД: scalar и scalars должны вернуть соответствующие
    значения.
17    fake_db = AsyncMock()
18    fake_scalars = MagicMock()

```



```

18     fake_scalars.all.return_value = [dummy_transcription]
19     fake_db.scalars = AsyncMock(return_value=fake_scalars)
20     fake_db.scalar = AsyncMock(return_value=1)
21
22     # Патчим функцию получения описания транскрипции
23     monkeypatch.setattr(
24         "source.app.transcriptions.services.get_transcription_descrt
25         iption",
26         AsyncMock(return_value="dummy description"),
27     )
28
29     result = await list_user_transcriptions(page=1, size=10, user_id=1,
30     db= fake_db)
31     assert result.total == 1
32     assert len(result.transcriptions) == 1
33     # Проверяем, что описание транскрипции заполнено
34     assert result.transcriptions[0].description == "dummy description"

```

4.5. Взятие статуса транскрипции

SUT: Transcriptions Service

(source.app.transcriptions.services.info_transcrip

Downstream: БД транскрипций

Детальный сценарий: Функция получает `user_id` и `transcript_id`, запрашивает запись транскрипции, проверяет право доступа (`creator_id`), и возвращает объект тран-скрипции с текущим `current_state`.

Сценарий 1 (успешный):

1. **Состояние до:** запись транскрипции с `id=55` существует, `creator_id=1`.
2. **Состояние после:** возвращён объект транскрипции, `current_state=IN_PROGRESS`.

Сценарий 2 (неуспешный — чужая транскрипция):

1. **Состояние до:** запись транскрипции существует, но `creator_id` $\neq$ `user_id`.
2. **Состояние после:** выбрасывается `ValueError("You may cancel processing only of your transcript")`.

```

1 async def test_get_transcription_status(monkeypatch):
2     from source.app.transcriptions.services import info_transcript
3
4     # Создаем фейковую транскрипцию с нужными данными
5     dummy_transcription = Transcription()
6     dummy_transcription.id = 55
7     dummy_transcription.creator_id = 1
8     dummy_transcription.current_state = TranscriptionState.IN_PROGRESS
9
10    # Мокаем метод получения транскрипции из БД (get_one)
11    fake_db = AsyncMock()
12    fake_db.get_one = AsyncMock(return_value=dummy_transcription)
13
14
15    result = await info_transcript(user_id=1, transcript_id=55,
16    db=fake_db) assert result.current_state == Transcript

```

5. Отчёт о прохождении интеграционных тестов

Данный раздел содержит результаты проведенного тестирования на сервере CI, в роли которого выступает Github Actions. Ниже представлены:

Общий пайплайн тестов

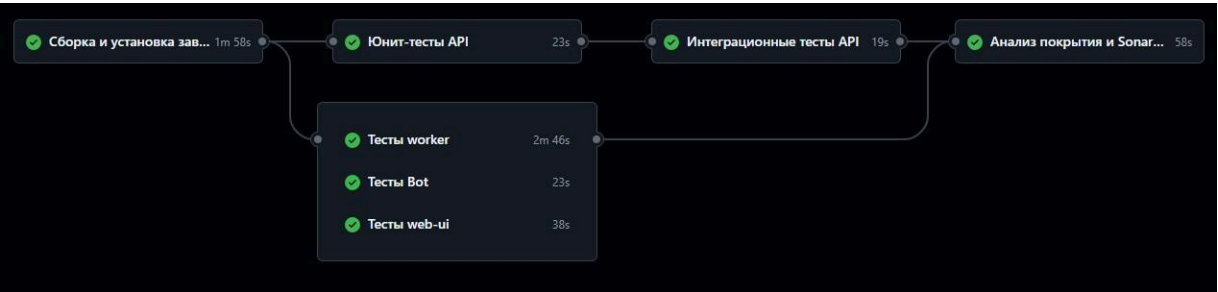


Рис. 1: Пайплайн запуска тестов.

```
1 Run pytest --rootdir=. --cov=source --cov-report=xml:api_integration_coverage.xml api/tests/integration_tests
2 ===== test session starts =====
3 platform linux -- Python 3.11.12, pytest-8.3.4, pluggy-1.5.0
4 rootdir: /home/runner/work/lay-lecture/lay-lecture
5 configfile: api/pytest.ini
6 plugins: cov-5.0.0, anyio-4.6.0, asynctest-0.18.0
7 asynctest: main-mode: sync, asynctest_default_fixture_loop_scope=function, asynctest_default_test_loop_scope=function
8 collected 5 items
9
10 api/tests/integration_tests/test_integration_api.py ..... [100%]
11
12 ===== warnings summary =====
13 ../usr/local/lib/python3.11.12/site-packages/pydantic/fields.py:1870
14 /opt/hostedtoolcache/Python/3.11.12/x64/lib/python3.11/site-packages/pydantic/fields.py:1870: PydanticDeprecatedSince20: Using extra keyword arguments on 'Field' is deprecated and will be removed, use 'json_schema_extra' instead. (Extra keys: 'env'). Deprecated in Pydantic V2.0 to be removed in V3.0. See
15 Pydantic V2 Migration guide at https://errors.pydantic.dev/2.11/migration/
16 warn()
17
18 ../usr/local/lib/python3.11.12/site-packages/passlib/utils/_init_.py:854
19 /opt/hostedtoolcache/Python/3.11.12/x64/lib/python3.11/site-packages/passlib/utils/_init_.py:854: DeprecationWarning: 'crypt' is deprecated and slated for removal in Python 3.13
20   from crypt import crypt as _crypt
21
22 -- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
23
24 ----- coverage: platform linux, python 3.11.12-final-0 -----
25 Coverage XML written to file api_integration_coverage.xml
26
27 ===== 5 passed, 2 warnings in 1.98s =====
```

Рис. 2: Выполнение тестов

Аналитика работы: получаемые логи позволяют определить, целый ряд факторов, а именно:

- 1. Корректную реакцию тестов на возникающие ошибки
- 2. Соответствие названий тестов запускаемым модулям и use-cases
- 3. Выполнение всех интеграционных тестов

6. Процедура расширения тестового набора

Процедура расширения тестового набора при добавлении новой функциональности

Процедура расширения тестового набора при добавлении новой функциональной части в интеграционном тестировании включает несколько этапов, направленных на обеспечение полного покрытия тестами вновь добавленного модуля.

1. Анализ требований:

На первом этапе изучаются требования к новой функциональной части. Необходимо определить, какие задачи решает модуль, какие входные и выходные данные использует, а также каким образом он взаимодействует с остальными компонентами системы.

2. Разработка тестовых сценариев:

На основании требований формируются тестовые сценарии, охватывающие как позитивные, так и негативные случаи использования. Особое внимание уделяется граничным условиям, а также взаимодействию с внешними сервисами и базами данных.

3. Интеграция тестов:

Новые тесты добавляются в существующий тестовый фреймворк, а также регистрируются в системе сборки и CI/CD. Это позволяет выполнять все тесты в рамках единого пайплайна и отслеживать влияние изменений на остальные модули.

4. Запуск и отладка тестов:

Выполняется первый запуск новых тестов. При необходимости производится отладка, исправление ошибок как в коде тестов, так и в бизнес-логике тестируемой функциональности.

5. Автоматизация тестов:

После верификации корректности выполнения тестов они подготавливаются для автоматизированного исполнения. Это включает стабилизацию окружения, настройку моков и использование вспомогательных утилит для повторного использования сценариев в будущем.